

Proyecto 1 – 2026 – Gestión de riesgos, excepciones y ALU multiciclo en MIPS

Serrano Fernández, Alberto

miércoles, 8 de abril de 2026

Breve resumen

En este proyecto se ha migrado un procesador MIPS uniciclo/segmentado base (con retardos y sin gestión de riesgos) a una arquitectura robusta con resolución de dependencias. Para ello, se ha diseñado la Unidad de Anticipación (UA) que soluciona riesgos de datos (RAW) mediante forwarding hacia la etapa EX, y la Unidad de Detención (UD) que inyecta burbujas (stalls) en IF/ID ante dependencias tipo load-use. Además, se han gestionado los riesgos de control invalidando instrucciones (flush) en saltos tomados. Se ha implementado un sistema completo de excepciones (IRQ, Data Abort, Undef), modificando el multiplexor del PC, añadiendo los registros status_reg y Exception_LR, e implementando el retorno con RTE. La ALU se ha modificado para incluir una FSM de 3 estados para instrucciones vectoriales MAC, gestionando el riesgo estructural con un stall_MIPS. Finalmente, se integraron contadores de rendimiento jerárquicos para monitorizar la eficiencia del código.

Verificación de los requisitos funcionales:

RF1 JAL y RET

Descripción: Incluir las instrucciones JAL y RET ya trabajadas en prácticas en el MIPS del proyecto

¿**Cómo?** En la Unidad de Control (UC), para JAL (op=000101): RegWrite=1, dirección de escritura=31 (fijo), el dato a escribir es PC+4 y selecciona la dirección de salto al PC. Para RET (op=000110): lee R31 y pasa ese valor directo al Mux del PC. Como son saltos, la UD debe forzar valid_I_IF=0.

Verificación: Comprobar su correcto funcionamiento en un banco de pruebas.

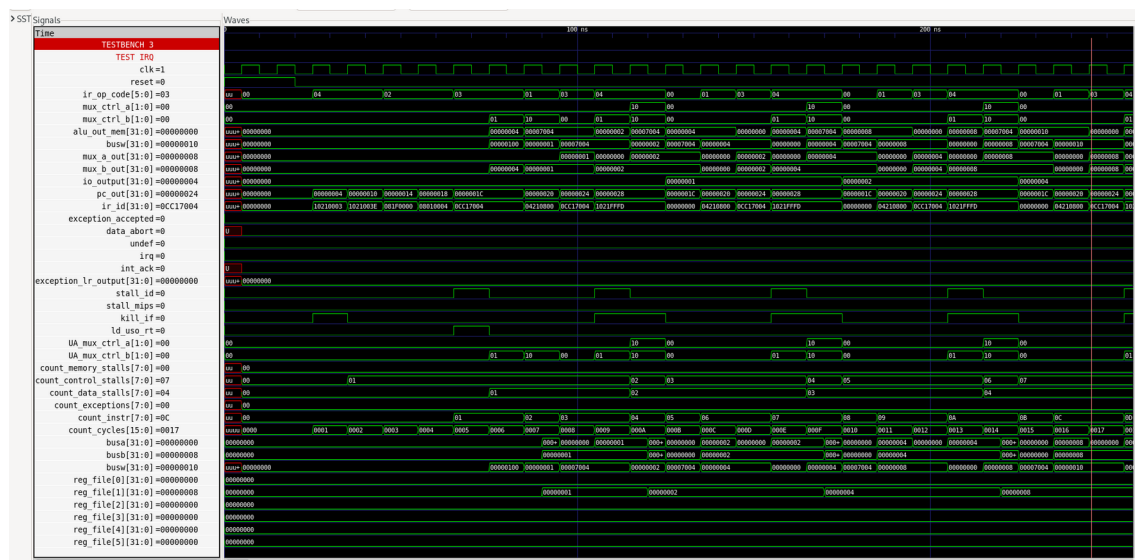
RF2 Anticipación de operandos

Descripción: El procesador es capaz de anticipar los operandos para las instrucciones LW, SW y ARIT a distancia 1 (siempre que el dato a anticipar no venga de un lw) y 2.

¿Cómo? La UA compara los registros destino de las etapas MEM WriteReg_MEM y WB WriteReg_WB con los registros fuente en EX Rs_EX y Rt_EX. Si coinciden y RegWrite está activo, se conmutan los multiplexores Mux_A_EX y o Mux_B_EX para tomar el dato adelantado. Prioridad siempre a MEM (distancia 1) sobre WB (distancia 2).

Verificación:

- En el banco de pruebas test_IRQ se realizan las siguientes anticipaciones:
 - De LW R1, 8(R0) a ADD r31, R1, R31: anticipación de Rs distancia 2 (tras detención de un ciclo del ADD)
 - De SUB r31, R31,R1 a LW R1, 0(R31): anticipación de Rs distancia 1.
 - De ADD R2, R1, R2 a SW R2, 7008(R6)



- Caso 4 Forwarding de JAL a consumidor en EX (distancia 2 desde WB)

JAL func

[burbuja flush] ; instrucción en IF se invalida

func: ADD R1, R31, R2

El flush de IF por el salto crea una burbuja, así que el destino de JAL llega a EX exactamente cuando JAL está en WB. En WB, busW = PC4_WB. El camino WB→EX lleva busW, por tanto la anticipación funciona sin hardware extra. JAL_uso_rs = '0' es correcto.

AOC2 - EINA - Universidad de Zaragoza

JAL escribe PC+4 en R31. El mux de escritura al BR selecciona PC4_WB cuando JAL_WB=1, por lo que busW = PC4_WB. El camino WB→EX del forwarding usa busW, así que la anticipación lleva el valor correcto.

No puede haber un consumidor con JAL en MEM porque el flush de IF inserta una burbuja: la instrucción destino llega a EX justo cuando JAL está en WB. El forwarding WB→EX lo cubre. No hace falta hardware extra ni stall. JAL_uso_rs = '0' es correcto.

- Caso 5 SW como consumidor de Rt (distancia 1)

ADD R3, R1, R2

SW R3, 0(R0)

SW no escribe en BR pero sí lee Rt. La UA detecta Reg_Rt_EX = RW_MEM. La anticipación de Rt funciona para SW.

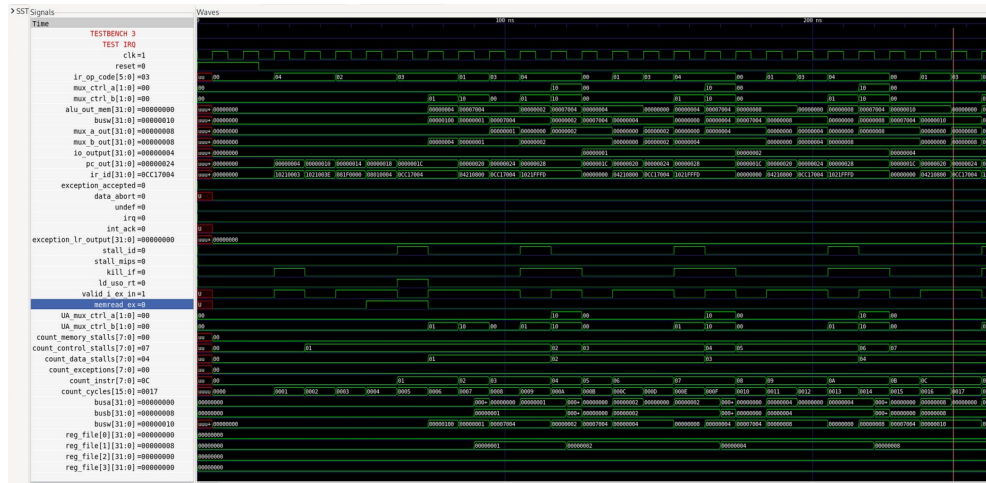
RF3 Riesgos de datos

Descripción: El procesador es capaz de detener la ejecución para evitar los riesgos causados por las dependencias en instrucciones lw-uso (distancia 1), beq o WRO (distancias 1 o 2).

¿**Cómo?** La UD detecta si la instrucción en EX es un load MemRead=1 y su destino coincide con los operandos fuente decodificados en ID. De ser así, levanta stall_ID=1. Esto congela el PC y el registro IF/ID, y pasa valid_I_ID=0 hacia EX. Para BEQ en ID, si depende de un cálculo en EX o MEM, también se para.

Verificación:

- En el banco de pruebas test_IRQ se realizan las siguientes detenciones:
 - o Test 1: 1 ciclo de detención por dependencia a distancia 2 entre ADD R1, R1, R1 y beq R1, R1, main.
 - o Test 2: 1 ciclo de detención por dependencia lw-uso en varios casos. Por ejemplo: de LW R1, 8(R0) a SW r1, 7004(r6)
 - o Test 3: No se para por riesgo de datos entre las instrucciones SW r1, 7004(r6) y ADD R1, R1, R1.



- Caso 4 — BEQ dependiente de instrucción en EX (distancia 1)

ADD R1, R1, R1

BEQ R1, R1, main

El test_IRQ ya lo hace. Escríbelo explícito: BEQ_rs='1' porque dep_rs_EX='1'
→ stall_ID=1.

- Caso 5 BEQ dependiente de instrucción en MEM (distancia 2)

ADD R1, R1, R1

NOP

BEQ R1, R1, main

- Caso 6 RET dependiente de instrucción en EX

LW R31, 0(R0)

RET

- Caso 7 SW NO genera stall

ADD R1, R2, R3

SW R1, 7004(R6)

RF4 Riesgos de control

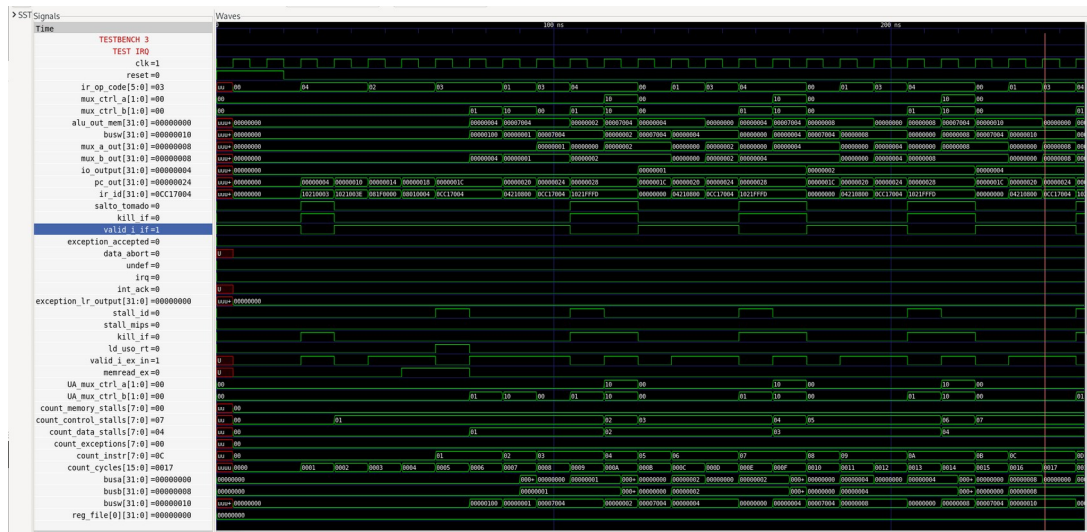
Descripción: El procesador es capaz eliminar los riesgos de control causados por las instrucciones de salto: BEQ y RTE.

¿Cómo? Ante un salto incondicional JAL, RET, RTE) o un BEQ donde se cumple la condición resuelto en ID, la UD activa un borrado lógico (flush) forzando a '0' el bit de validez del registro IF/ID (valid_I_IF).

Verificación:

AOC2 - EINA - Universidad de Zaragoza

- En el banco de pruebas test_IRQ se realizan varios saltos tomados en los que sólo se ejecutan las instrucciones adecuadas:
 - o beq R1, R1, main
 - o rte



- Caso 3 JAL siempre hace flush de IF
JAL func
- Caso 4 — RET siempre hace flush de IF
RET ; salto incondicional → Kill_IF=1 siempre
JAL, RET y RTE activan salto_tomado incondicionalmente por tanto siempre flush.

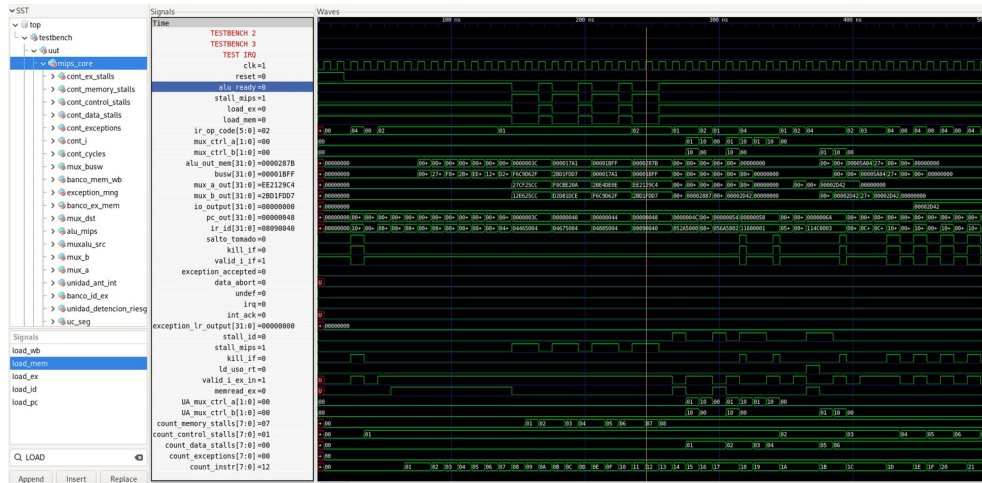
RF5 Riesgos estructurales

Descripción: El procesador es capaz de detener la ejecución para evitar riesgos estructurales causados por las instrucciones MAC y MAC_ini.

¿Cómo? La UD vigila la señal MAC_busy de la ALU. Si está activa, levanta la señal general stall_MIPS. Esta señal paraliza todo el pipeline completo y bloquea la aceptación de interrupciones mientras la ALU multiciclo termina.

Verificación:

- En el banco de pruebas testbench_2 se ejecutan varias instrucciones MAC y MAC_ini, incluyendo:
 - o mac_ini R10, R1, R5
 - o mac R10, R2, R6



RF6 Cambio a modo excepción

Descripción: Si el procesador está en modo usuario y recibe IRQ, ABORT o UNDEF pasa al modo correspondiente y ejecuta la rutina que indica la tabla de vectores de excepción. Si está en modo excepción ignora las señales hasta volver a modo usuario.

¿Cómo? Si status_reg="00" y llega una excepción válida, el hardware inyecta en el PC el vector 0x4 IRQ, 0x8 Abort, 0xC Undef. Se fuerza un flush de IF, ID y EX (valid=0). El PC de la instrucción invalidada más antigua se almacena en Exception_LR. El registro de estado pasa a modo excepción status_reg="11".

Verificación:

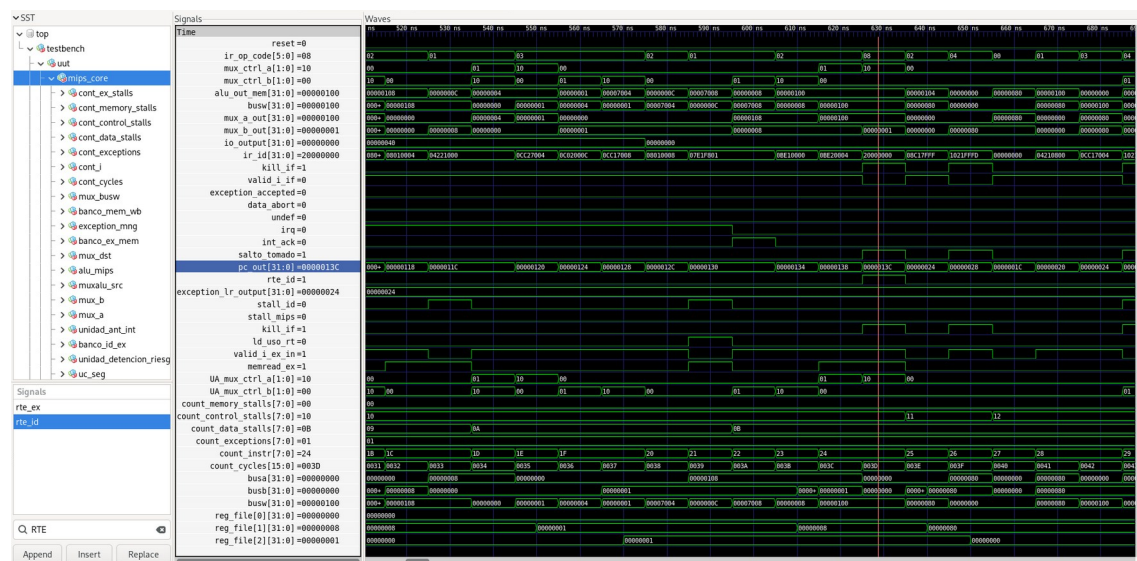
- IRQ: banco de pruebas test_IRQ en el que se realizan diversas IRQs. Todas ellas se atienden si y sólo si el procesador está en modo usuario. Se ejecuta una RTI que contabiliza el número de IRQs. Se verifica su funcionamiento correcto.
- Data Abort: en la memoria de instrucciones se incluyen dos bancos de pruebas en el que se realizan un acceso no alineado, y un acceso a una dirección fuera de rango. En ambos casos la ejecución salta a la rutina Data Abort (bucle infinito).
- Undef: en la memoria de instrucciones se incluye un banco de pruebas en el que se ejecuta una instrucción con un código de instrucción desconocido. La ejecución salta a la rutina UNDEF (bucle infinito).

RF7 Retorno a modo usuario RTE

Descripción: Al terminar la excepción se vuelve a la ejecución original con la instrucción RTE

¿Cómo? En ID, RTE conmuta el multiplexor del PC para que la próxima instrucción se traiga desde Exception_LR. Modifica el status_reg forzándolo a "00" rehabilitando IRQs. Realiza un flush de la instrucción que venía detrás de ella en IF.

Verificación: Sólo tenéis que verificarlo para las IRQs. El código debe funcionar aunque se interrumpa repetidas veces. Para el resto de excepciones se asume que es un fallo irresoluble y se debe resetear el sistema, pero la gestión es idéntica que para las IRQs. En el banco de pruebas test_IRQ se puede comprobar que se retorna a la ejecución sin alterarla siempre que se siga un esquema de prólogo y epílogo apropiado (guardando y restaurando los registros en pila). Para ello se utiliza el registro 31 como SP.



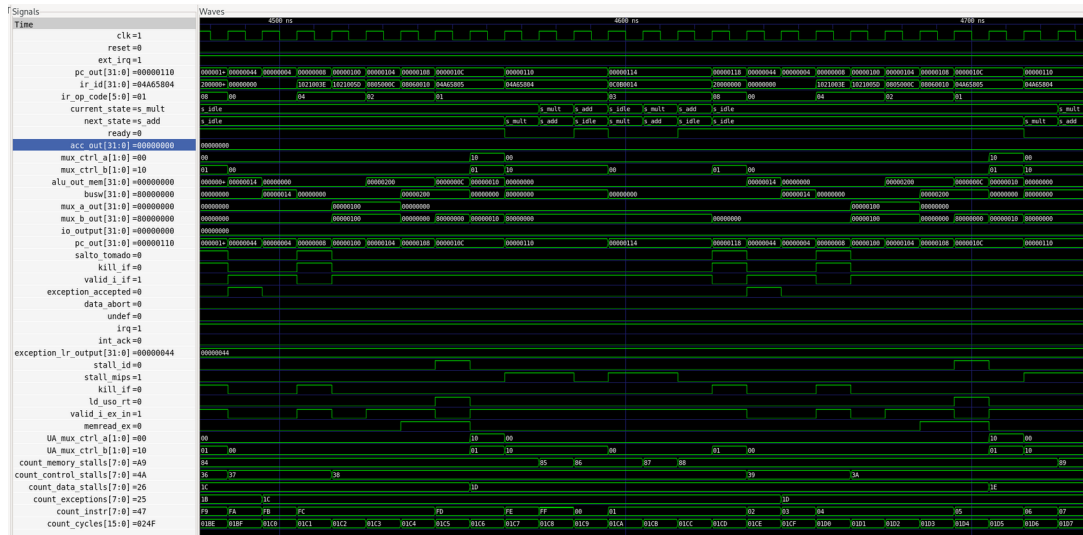
RF8 ALU multiciclo

Descripción: El procesador es capaz de ejecutar las instrucciones MAC y MAC_ini en 3 ciclos y gestionar el estado del registro ACC ante el tratamiento de excepciones.

¿Cómo? FSM de 3 estados: S_IDLE (registra productos, ready=0), S_MULT (registra suma en árbol, ready=0), y S_ADD (acumula en ACC, ready=1 liberando el stall de la UD). La gestión del ACC en excepciones es por software: la rutina de interrupción debe guardarlo en pila al entrar y restaurarlo antes del RTE.

Verificación:

- Diseñado testbench_6 (en fichero RAM_I_test_exceptions_neuron_2026.vhd)



- se observa la ejecución de una instrucción vectorial donde la máquina de estados de la ALU completa exitosamente sus tres ciclos lógicos

RF9 Contadores

Descripción: los contadores nos dan información de la ejecución del código

¿Cómo? Se utiliza lógica combinacional directa sobre las señales de la UD y del pipeline:

Instrucciones dinámicas: Se incrementa directamente con valid_I_WB.

Riesgos de control: Incrementan cuando se anula la instrucción en IF (kill_IF).

Riesgos de datos: Incrementan con stall_ID, pero se condicionan con not(stall_MIPS) para darle menor jerarquía y no sumar si el procesador está bloqueado por memoria o ALU.

Riesgos estructurales: Se desglosan evaluando stall_MIPS.

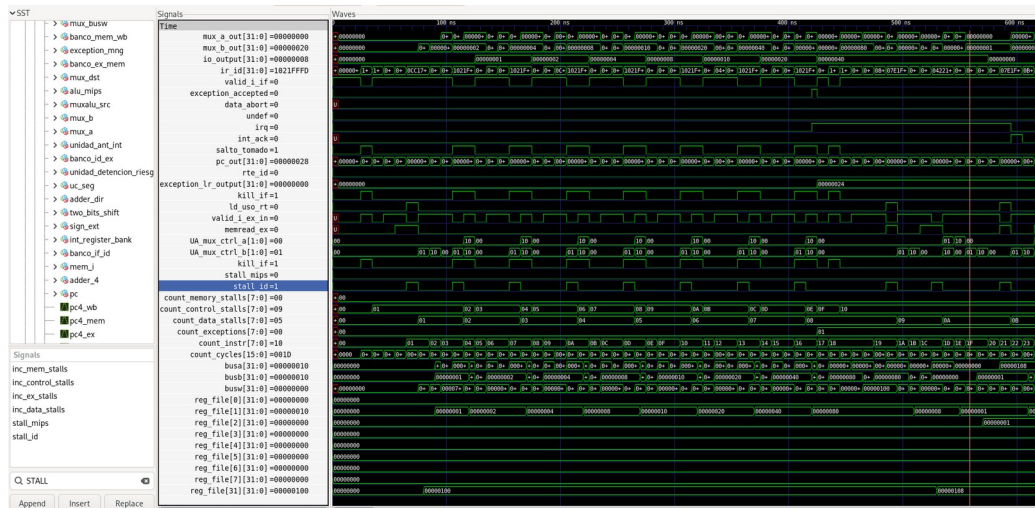
Excepciones: Se cuenta directamente el pulso de Exception_accepted.

Verificación: En el banco de pruebas test_IRQ se ha comprobado que:

- Por cada instrucción válida que hace su etapa WB se incrementa el contador de instrucciones Ins. Si la instrucción no es válida no se incrementa.
- Data_stalls contabilizan bien los ciclos debidos a los riesgos de datos incrementando el número indicado en los test mencionados en el RF5.
- Control_stalls contabilizan bien los ciclos debidos a los riesgos de control, incrementando un ciclo en cada salto tomado (ver pruebas en el RF6).
- Exception_accepted contabiliza las excepciones aceptadas. Su cuenta coincide con la cuenta que realiza el propio código.

AOC2 - EINA - Universidad de Zaragoza

- **Memory_stalls:** cada vez que se pide una operación a memoria, y esta no esta “ready” se incrementa este contador.
- **IMPORTANTE:** las paradas deben incrementar sólo un contador. Si hay hay varios motivos, se cuenta el de mayor jerarquía : `memory stall> data_stall> control_stall`. Ya que el otro se mantendrá y se contabilizará cuando toque.



Cuantificación de horas dedicadas

- Estudio del MIPS, VHDL, entorno, instalación...: 3h
- Adición de las nuevas instrucciones: 2h
- Gestión de excepciones: 4h
- Gestión de riesgos: 6h
- Depuración, verificación y programas de prueba: 8h
- Memoria: 2h

Conclusiones y Autoevaluación

El trabajo ha servido para entender en la práctica cómo las dependencias teóricas del código afectan al hardware real. Pasar de un procesador retardado ideal a uno realista con resolución activa de riesgos UD/UA y soporte multiciclo. El proyecto ha sido exigente en depuración pero muy instructivo a nivel de herramientas (GTKWave/GHDL).